

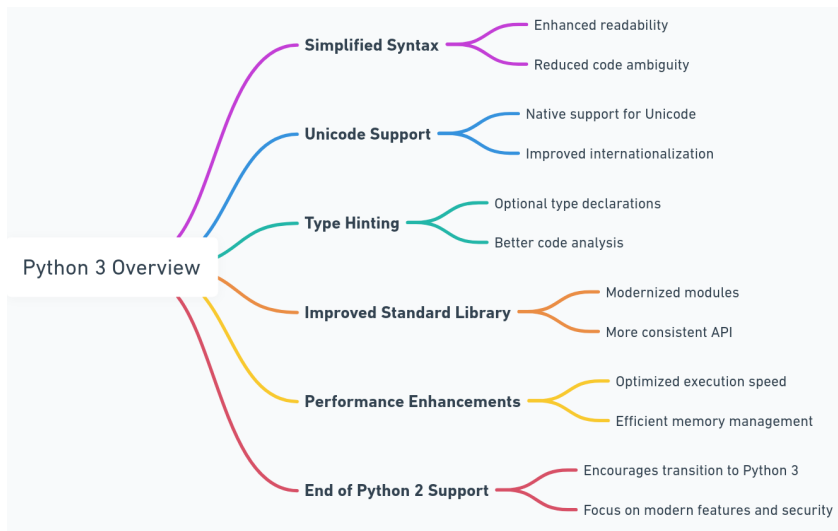


Figure 1: Python 3 overview

sequences of bytes. This fundamental shift greatly simplifies text processing and internationalisation, making Python 3 more suitable for modern applications that deal with multilingual content.

Python 2: Strings can be stored as Unicode using the 'u' in front of them.

```
unicode_string = u"Hello, World! "
print unicode_string
```

Python 3:

```
unicode_string = "Hello, World!"
print(unicode_string)
```

Syntax enhancements: Python 3 introduces several syntax enhancements and clarifications, enhancing code readability and consistency. Notable changes include the replacement of the `print` statement with the `print()` function, ensuring uniformity in code structure. Additionally, integer division now returns a float by default, avoiding common pitfalls associated with integer division in Python 2.

- **Print statement vs print function:**
Python 2:

```
print "Hello, World!"
```

Python 3:

```
Print ("Hello, World!")
```

Type hints: Type hints, introduced in Python 3, allow developers to annotate function arguments and return types, aiding in code understanding and maintenance. While these hints are not enforced by the interpreter, they enable static analysis tools to catch potential type-related errors early in the development process, promoting code reliability and maintainability.

Python 2:

```
# Type hints not supported
```

Python 3:

```
def add_numbers(a: int, b: int) -> int:
    return a + b
result = add_numbers(3, 5)
print("sum:", result)

sum: 8
```

Extended iterators and generators:

Python 3 enhances iterator and generator capabilities with features like the `yield` from syntax, simplifying the

composition and delegation of iterable objects. These enhancements make it easier to write clean, concise, and efficient code, particularly when dealing with complex data processing tasks or large datasets.

- **Iteration:**

Python 2: In Python 2, the `range` function creates a list of numbers and `xrange()` creates a generator.

```
for i in xrange(5):
    print (i)
```

Python 3: In Python 3, `xrange` becomes `range` and `range` of Python 2 can be used as `list(range)`.

```
for i in range(5):
    print (i)
```

Enhanced standard library:

Python 3 ships with an expanded and improved standard library, featuring updated modules and removing obsolete or redundant components. This modernisation ensures that developers have access to a rich set of tools and utilities for various development tasks, ranging from networking and file I/O to data manipulation and web development.

Asyncio for asynchronous programming: The introduction of the `asyncio` module in Python 3 facilitates asynchronous programming, enabling developers to write highly scalable and efficient I/O-bound applications. By leveraging coroutines and event loops, Python 3 empowers developers to build responsive and concurrent systems with ease, enhancing application performance and responsiveness.

Integer division: Python 2: In Python 2, integer division truncates the result towards zero, so `7 / 3` would result in 2.

```
division_result = 7 / 3
print division_result    #Outputs:
2
```